

# Engineering Knowledge Assistant

## Project Summary & Engineering Record

|                   |                                                                                                                                   |
|-------------------|-----------------------------------------------------------------------------------------------------------------------------------|
| Project Status    | Functional portfolio prototype                                                                                                    |
| Primary Domain    | AI Engineering / RAG / Document Intelligence                                                                                      |
| Core Technologies | Python, OpenAI, Qdrant, SQLite, Streamlit                                                                                         |
| Repository        | <a href="https://github.com/JacklynInc/Engineering-Knowledge-Assistant">github.com/JacklynInc/Engineering-Knowledge-Assistant</a> |

**Purpose:** A detailed record of the system architecture, engineering decisions, implementation challenges, evaluation, skills demonstrated, and final portfolio milestone.

## 1. Project Overview

The **Engineering Knowledge Assistant** is a Python-based AI application that allows users to upload engineering manuals, SOPs, and technical documents and ask questions about their contents in natural language.

The application uses **Retrieval-Augmented Generation (RAG)** to retrieve relevant document sections before generating an answer. This grounds responses in the uploaded engineering material.

The system preserves document and page metadata so retrieved information can be traced to its source.

The project combines Python, AI/LLMs, RAG, vector databases, relational databases, PDF processing, semantic search, Streamlit, and Git/GitHub.

## 2. Problem Statement

Engineering professionals frequently work with large technical documents such as equipment manuals, Standard Operating Procedures, technical reports, maintenance documentation, engineering specifications, research documents, and operating instructions.

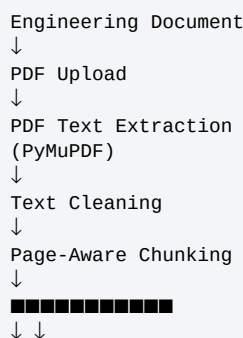
Finding specific information manually can be time-consuming.

The goal was to build an AI-powered knowledge assistant that lets users upload a document, process and index it, ask natural-language questions, retrieve relevant sections, generate grounded answers, and identify supporting source pages.

### 3. Project Objectives

- Build an end-to-end RAG application.
- Process engineering PDF documents automatically.
- Preserve document and page metadata.
- Store structured document information in SQLite.
- Generate semantic embeddings using the OpenAI API.
- Store and search embeddings using Qdrant.
- Retrieve relevant engineering information using semantic similarity.
- Generate answers grounded in retrieved document context.
- Provide page-aware source references.
- Build a usable Streamlit interface.
- Evaluate retrieval and RAG behaviour.
- Practice modular Python application development.
- Apply software engineering principles to an AI application.

## 4. System Architecture



```
SQLite OpenAI Embeddings
↓
Qdrant
↓
Semantic Retrieval
↓
Relevant Chunks
↓
Context Assembly
↓
LLM Generation
↓
Grounded Answer + Sources
↓
Streamlit Interface
```

## 5. End-to-End RAG Pipeline

### Step 1 — Document Upload

The user uploads an engineering PDF through the Streamlit interface.

### Step 2 — PDF Text Extraction

PyMuPDF extracts text from each page while preserving page numbers.

### Step 3 — Text Cleaning

Extracted text is cleaned before further processing.

### Step 4 — Chunking

The document is divided into smaller page-aware text chunks retaining document ID, chunk number, page number, chunk text, and character count.

### Step 5 — Relational Storage

Document and chunk metadata are stored in SQLite.

### Step 6 — Embedding Generation

Each chunk is converted into a vector embedding using text-embedding-3-small.

### Step 7 — Vector Storage

Embeddings are stored in Qdrant using cosine similarity for semantic search.

### Step 8 — User Question

The user asks a question in natural language.

### Step 9 — Query Embedding

The question is converted into an embedding using the same embedding model.

### Step 10 — Semantic Retrieval

Qdrant searches for semantically similar chunks, optionally restricted to the selected document.

### Step 11 — Context Construction

The most relevant retrieved chunks are assembled into a context window.

### Step 12 — LLM Generation

The retrieved context and question are passed to the language model, which is instructed to answer using the supplied context.

## Step 13 — Source Attribution

The application displays document name, page number, chunk number, similarity score, and retrieved text.

## 6. Technology Stack

### Programming

- Python 3.14.4
- SQLite
- Streamlit

### AI and RAG

- OpenAI API
- text-embedding-3-small
- GPT-5.6 Luna
- Retrieval-Augmented Generation
- Vector embeddings
- Semantic search
- Prompt-based grounding

### Vector Database

- Qdrant

### Document Processing

- PyMuPDF
- PDF text extraction
- Text cleaning
- Page-aware chunking

### Development Tools

- Visual Studio Code
- Git
- GitHub
- Python virtual environment

## 7. Project Structure

```
Engineering-Knowledge-Assistant/  
■■■ data/  
■ ■■■ pdfs/  
■ ■■■ database/  
■ ■■■ embeddings/  
■ ■■■ exports/  
■■■ screenshots/  
■ ■■■ assistant-demo.png  
■ ■■■ assistant-demo1.png  
■■■ src/  
■ ■■■ pdf_reader.py  
■ ■■■ text_cleaner.py  
■ ■■■ chunker.py  
■ ■■■ database.py
```

```
■ ■■■ embedding.py
■ ■■■ vector_database.py
■ ■■■ rag.py
■ ■■■ llm.py
■ ■■■ ingestion.py
■■■ tests/
■ ■■■ test_database_transaction.py
■ ■■■ test_rag.py
■ ■■■ test_retrieval.py
■ ■■■ test_vector_cleanup.py
■■■ app.py
■■■ main.py
■■■ index_vectors.py
■■■ README.md
■■■ PROJECT_SUMMARY.md
■■■ .env.example
■■■ .gitignore
```

## 8. Module Responsibilities

### pdf\_reader.py

Extracts text from PDF documents while preserving page numbers.

### text\_cleaner.py

Performs basic normalization of extracted document text.

### chunker.py

Divides page content into manageable text chunks while preserving page metadata.

### database.py

Creates SQLite tables, stores and retrieves documents/chunks, deletes records, and manages transactional document ingestion.

### embedding.py

Generates OpenAI embeddings for document chunks and user queries.

### vector\_database.py

Creates the Qdrant collection, stores embeddings, performs semantic search, filters retrieval by document, and removes vectors.

### ingestion.py

Coordinates PDF extraction, cleaning, chunking, SQLite storage, embedding generation, and Qdrant indexing, with cleanup on failure.

### rag.py

Coordinates query embedding, Qdrant retrieval, SQLite chunk lookup, context construction, LLM generation, and source reporting.

### llm.py

Communicates with the OpenAI Responses API and generates answers grounded in retrieved document context.

### app.py

Provides the Streamlit interface for document selection, PDF upload, question input, answer display, source display, and similarity scores.

## 9. Database Design

## Documents

Stores document-level information.

```
documents
■■■ id
■■■ file_name
■■■ file_path
■■■ page_count
■■■ character_count
■■■ upload_date
```

## Chunks

Stores individual document sections.

```
chunks
■■■ id
■■■ document_id
■■■ chunk_number
■■■ page_number
■■■ chunk_text
■■■ character_count
```

The relationship is one document to many chunks.

```
Document
■
■■■ Chunk 1
■■■ Chunk 2
■■■ Chunk 3
■■■ ...
```

## 10. Vector Database Design

Qdrant stores the semantic representation of document chunks.

Each vector contains metadata such as `document_id`, `chunk_number`, and `page_number`.

- Collection: `engineering_chunks`
- Vector size: 1536
- Distance metric: Cosine similarity

This allows natural-language questions to be matched against document content based on semantic similarity rather than exact keyword matching.

## 11. Key Engineering Decisions

### SQLite + Qdrant

SQLite handles structured document and chunk metadata, while Qdrant handles vector similarity search.

### Page-Aware Chunks

Each chunk retains its original page number, making retrieved information traceable to the source.

### Document-Aware Retrieval

Semantic searches can be restricted to the selected document.

### Transaction-Safe Database Ingestion

Document and chunk insertion can be performed within a single SQLite transaction. Failures trigger rollback.

### Vector Cleanup

If embedding generation or vector insertion fails after database insertion, the associated document and vectors are removed.

## Grounded LLM Generation

The model receives retrieved context and is instructed to answer using that context without inventing technical information or page numbers.

## Cost-Conscious API Usage

Embeddings are generated during ingestion. Question-time API usage is limited to query embedding and final answer generation.

## 12. Error Handling and Data Consistency

The ingestion pipeline includes protection against partial processing.

```
Database insertion
↓
Embedding generation
↓
Vector insertion
↓
SUCCESS
```

If vector processing fails:

```
Vector processing failure
↓
Delete vectors
↓
Delete document
↓
Cleanup
```

This prevents incomplete knowledge-base entries and reduces inconsistency between SQLite and Qdrant.

## 13. Evaluation and Testing

### Retrieval Evaluation

tests/test\_retrieval.py tests relevant questions, expected source pages, wrong-document questions, unrelated questions, and similarity thresholds.

### RAG Evaluation

tests/test\_rag.py tests relevant questions, generated answers, source pages, no-information behaviour, and unrelated questions.

### Database Transaction Test

tests/test\_database\_transaction.py tests document insertion, chunk insertion, database retrieval, and cleanup.

### Vector Cleanup Test

tests/test\_vector\_cleanup.py tests vector deletion functionality used during document cleanup.

## 14. Evaluation Results

The retrieval evaluation produced successful results for the implemented test cases.

## 15. Engineering Challenges Solved

### Python Environment

Resolved virtual-environment configuration issues by recreating the environment cleanly.

### PyMuPDF API Changes

Updated the PDF reader to use `import pymupdf`, removing the older deprecation warning.

## SQLite Document Lifecycle

Implemented careful document/chunk insertion, deletion, and transaction rollback.

## Qdrant Filtering

Corrected Qdrant filter construction for document-specific retrieval.

## Qdrant Local Persistence

Distinguished client lifecycle/shutdown warnings from actual application failures.

## OpenAI API Authentication

Regenerated the API key and corrected environment configuration while keeping secrets outside Git.

## OpenAI API Billing

Configured API billing so embedding generation could be tested successfully.

## Streamlit State

Handled uploaded files, existing documents, reruns, document selection, and question submission.

## Git and GitHub

Established version control and pushed the documented application to GitHub.

# 16. Security and Configuration

The actual OpenAI API key is stored locally in `.env`.

The `.env` file is excluded from Git through `.gitignore`.

A safe configuration example is provided through `.env.example`:

```
OPENAI_API_KEY=your_api_key_here
```

Sensitive local files, databases, and generated data are excluded from the public repository through `.gitignore`.

# 17. Cost Optimization Approach

- Use local SQLite for structured storage.
- Use local Qdrant for vector storage.
- Generate embeddings during ingestion.
- Reuse stored embeddings.
- Use retrieval before generation.
- Avoid LLM calls during basic application testing.
- Use deterministic and local tests where possible.
- Use a smaller embedding model.
- Keep retrieved context controlled.

This provides a practical balance between AI capability and API cost.

# 18. Skills Demonstrated

## Python Development

- Modular Python architecture
- Functions and data structures
- File processing
- Exception handling
- Database integration
- Virtual environments
- API integration

## **AI Engineering**

- LLM integration
- Embeddings
- Retrieval-Augmented Generation
- Prompt engineering
- Context construction
- Grounded generation
- Semantic retrieval
- Similarity search

## **Data Engineering**

- SQLite
- Relational data modelling
- Metadata management
- Database transactions
- Data cleanup
- Vector databases

## **AI Application Development**

- PDF ingestion
- Semantic search
- Retrieval pipelines
- RAG pipelines
- Streamlit application development
- AI evaluation

## **Software Engineering**

- Git
- GitHub
- Testing
- Debugging
- Documentation
- Separation of concerns

- Error handling
- Data consistency
- Configuration management

## 19. What This Project Demonstrates

This project demonstrates practical experience building an AI application rather than simply consuming an AI tool.

```
Engineering Domain Knowledge
+
Python
+
Document Processing
+
Databases
+
Vector Search
+
RAG
+
LLMs
+
Application Development
```

The result is a working engineering-focused AI knowledge system.

## 20. Current Limitations

- No OCR for scanned documents
- Basic page-aware chunking
- No hybrid keyword/vector retrieval
- No dedicated reranking model
- Local Qdrant storage
- No authentication
- No cloud deployment
- Limited automated evaluation dataset
- No production monitoring
- Single-user Streamlit workflow

These are potential future improvements rather than blockers to the current project milestone.

## 21. Future Improvements

### Document Intelligence

- OCR for scanned PDFs
- Table extraction
- Image and diagram extraction
- Better document structure detection
- Section-aware chunking

### Retrieval

- Hybrid keyword + vector search

- Query expansion
- Reranking
- Improved metadata filtering
- Adaptive retrieval thresholds

## RAG

- Better context selection
- Context compression
- Citation validation
- Automated hallucination evaluation
- Multi-document reasoning

## Infrastructure

- Docker
- Cloud deployment
- API backend
- Authentication
- Persistent hosted vector database
- Production monitoring

## Evaluation

- Larger evaluation datasets
- Retrieval precision and recall
- Answer faithfulness
- Citation accuracy
- Automated regression testing

# 22. Portfolio Relevance

The project demonstrates the ability to build an AI-powered engineering application from the document ingestion layer through semantic retrieval, LLM generation, evaluation, and user interface development.

**Engineering knowledge + Python + AI + RAG + databases + vector search + application development.**

The project is particularly relevant to roles involving:

- AI Engineering
- Python Development
- AI Automation
- RAG Systems
- Document Intelligence
- Technical Software
- Industrial AI
- Engineering Data
- Applied AI

- Intelligent Automation

## 23. Project Milestone

```
Upload
↓
Process
↓
Store
↓
Embed
↓
Index
↓
Retrieve
↓
Generate
↓
Cite
↓
Display
```

The project has reached a stable portfolio milestone and is intentionally being frozen before adding more advanced production features.

## 24. GitHub Repository

**Repository:** <https://github.com/JacklynInc/Engineering-Knowledge-Assistant>

The repository includes:

- Source code
- Test suite
- README documentation
- Project summary
- Screenshots
- Environment configuration example
- Git history

Sensitive configuration files and local databases are excluded through .gitignore.

## 25. Final Project Outcome

The Engineering Knowledge Assistant successfully evolved from a basic document-processing concept into a functional AI engineering application.

The completed system demonstrates the complete lifecycle of a practical RAG application:

```
Engineering Document
↓
Document Processing
↓
Structured Storage
↓
Vector Embeddings
↓
Semantic Retrieval
↓
Context Grounding
↓
LLM Generation
↓
Source Attribution
↓
```

User Interface  
↓  
Evaluation

The project provided practical experience across AI engineering, Python development, document processing, vector databases, relational databases, RAG architecture, API integration, testing, debugging, Git/GitHub, and application development.

## 26. Project Completion Checklist

- Project concept defined
- Python environment configured
- Git repository created
- GitHub repository created
- PDF extraction implemented
- Text cleaning implemented
- Chunking implemented
- SQLite database implemented
- OpenAI embeddings integrated
- Qdrant vector database integrated
- Semantic retrieval implemented
- RAG pipeline implemented
- LLM answer generation implemented
- Source/page retrieval implemented
- Streamlit interface implemented
- Document upload implemented
- Retrieval evaluation implemented
- RAG evaluation implemented
- Database transaction testing implemented
- Vector cleanup testing implemented
- README completed
- Screenshots added
- .env.example added
- .gitignore configured
- Application UI polished
- Changes committed to Git
- Changes pushed to GitHub

## 27. Key Takeaway

The main achievement of this project is not simply building a chatbot.

It is the ability to design and implement an end-to-end **AI-powered engineering information system** in which document processing, structured storage, vector search, retrieval, LLM generation, source attribution, testing, and user

interaction work together as a single application.

**Project status: COMPLETE — Portfolio Milestone Reached.**